



UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS
DEPARTAMENTO DE CIENCIAS EXACTAS - DCEX-SS
CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PERIODO : Abril – Agosto 2025

ASIGNATURA : Programación Orientada a Objetos

TEMA : Código Limpio

NOMBRE : Nuñez Mishelle

NIVEL-PARALELO : Primero A

DOCENTE : Ing. Jhon Cruz

FECHA DE ENTREGA : Miércoles 30 de abril de 2025

SANTO DOMINGO - ECUADOR

2025

CONTENIDO

INTRODUCCIÓN.....	3
OBJETIVOS.....	3
- Objetivo General	3
- Objetivos Específicos.....	3
DESARROLLO	3
CONCLUSIONES.....	7
ANEXOS	8
BIBLIOGRAFÍA.....	11

INTRODUCCIÓN

El libro *Código Limpio: Un manual de artesanía del software* del autor Robert C. Martin es una obra de referencia en el ámbito del desarrollo de software, especialmente en lo que respecta a las buenas prácticas de programación orientadas a la legibilidad, mantenibilidad y calidad del código. En particular, los capítulos 2 y 3 abordan principios fundamentales que todo desarrollador debe dominar: la elección de nombres significativos y el diseño adecuado de funciones.

En el capítulo 2, titulado *Nombres significativos*, el autor subraya la importancia de utilizar nombres claros, descriptivos y precisos en variables, métodos, clases y otros elementos del código. Un buen nombre debe comunicar la intención del elemento al que representa, evitando ambigüedades, abreviaturas crípticas o denominaciones genéricas. Esta práctica no solo mejora la comprensión del código por parte de otros desarrolladores, sino que también reduce la necesidad de comentarios innecesarios.

Por su parte, el capítulo 3, denominado *Funciones*, se centra en la creación de funciones que sean breves, coherentes y que realicen una única tarea. Según Martin, una función ideal debe ser fácil de leer, poseer un nombre que refleje exactamente su propósito y minimizar tanto la cantidad de parámetros como la presencia de efectos secundarios. La correcta estructuración de funciones permite escribir software más limpio, modular y fácil de mantener.

Ambos capítulos refuerzan la premisa de que el código fuente no es únicamente una instrucción para la máquina, sino una forma de comunicación entre personas. Por tanto, adoptar estas prácticas es esencial para el trabajo colaborativo y la sostenibilidad del desarrollo a largo plazo.

OBJETIVOS

- Objetivo General

Mejorar la calidad del código mediante un enfoque sistemático en la limpieza, claridad y coherencia del mismo.

- Objetivos Específicos

- Identificar los errores planteados en el código ejemplo.
- Corregir la codificación en base a los capítulos dos y tres del libro “CODIGO LIMPIO ...” del autor Robert C. Martin.

DESARROLLO

- Aplicación del Capítulo dos del libro “CODIGO LIMPIO ...” del autor Robert C. Martin en el siguiente ejemplo:

```
import java.util.Scanner;
```

```
public class BadNames {
```

```
// Arreglo donde guardaremos hasta 10 números
private int[] n = new int[10]; // ¿qué es "n"?
private int i = 0;           // índice actual... ¿"i" de qué?

// Método "a": agrega un número al arreglo
public void a(int v) {
    n[i++] = v;           // "v" tampoco dice mucho
}

// Método "b": calcula la suma
public int b() {
    int s = 0;           // "s" -> ¿suma? ¿salario?
    for (int j = 0; j < i; j++) {
        s += n[j];
    }
    return s;
}

// Método "c": promedio de los números guardados
public double c() {
    return i == 0 ? 0 : (double) b() / i;
}

// Pequeña interfaz de consola para ejecutar y probar
public static void main(String[] args) {
    BadNames x = new BadNames();           // ¿por qué "x"?
    Scanner sc = new Scanner(System.in);

    System.out.print("¿Cuántos números ingresará? ");
    int t = sc.nextInt();           // "t" -> ¿total?

    for (int k = 0; k < t; k++) {           // "k" -> contador genérico
        System.out.print("Número: ");
        x.a(sc.nextInt());
    }

    System.out.println("Suma = " + x.b());
    System.out.println("Promedio = " + x.c());
    sc.close();
}
}
```

- Código modificado en base al capítulo dos.

```
package com.mycompany.badnames;
```

```
import java.util.Scanner;
```

```
/**
```

```
 * @author Mishelle Nuñez
```

```
 */
```

```
public class BadNames {
```

```
    // Arreglo donde guardaremos hasta 10 números
```

```
    private int[] numero = new int[10];    // Arreglo de numero hasta 10
```

```
    private int contador = 0;    // contador de numeros agregados
```

```
    // Método: agrega un número al arreglo
```

```
    public void agregar(int valor) {  
        numero[contador++] = valor;  
    }
```

```
    // Método: calculo de la suma
```

```
    public int suma() {  
        int sumar = 0;  
        for (int j = 0; j < contador; j++) {  
            sumar += numero[j];  
        }  
        return sumar;  
    }
```

```
    // Método: promedio de los números guardados
```

```
    public double promedio() {  
        return contador == 0 ? 0 : (double) suma() / contador;  
    }
```

```
    // Pequeña interfaz de consola para ejecutar y probar
```

```
    public static void main(String[] args) {  
        BadNames numeros = new BadNames();  
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("Cuantos numeros ingresara? ");
```

```
        int total = scanner.nextInt();
```

```
        for (int k = 0; k < total; k++) {    // "k" -> contador genérico
```

```
            System.out.print("Numero: ");
```

```
            numeros.agregar(scanner.nextInt());
```

```
        }
```

```
        System.out.println("Suma = " + numeros.suma());
        System.out.println("Promedio = " + numeros.promedio());
        scanner.close();
    }
}
```

- **Código modificado en base al capítulo tres.**

```
package com.mycompany.badnames;

import java.util.Scanner;

/**
 * @author Mishelle Nuñez
 */
public class BadNames {

    // Arreglo donde guardaremos hasta 10 números
    private int[] numero = new int[10]; // Arreglo de numero hasta 10
    private int contador = 0;           // contador de numeros agregados

    // Método: agrega un número al arreglo
    public void agregar(int valor) {
        numero[contador++] = valor;
    }

    // Método: calculo de la suma
    public int suma() {
        int sumar = 0;
        for (int j = 0; j < contador; j++) {
            sumar += numero[j];
        }
        return sumar;
    }

    // Método: promedio de los números guardados
    public double promedio() {
        return contador == 0 ? 0 : (double) suma() / contador;
    }

    // Pequeña interfaz de consola para ejecutar y probar
    public static void main(String[] args) {
        BadNames numeros = new BadNames();
    }
}
```

```
Scanner scanner = new Scanner(System.in);

int cantidad = pedirCantidad(scanner);
leerNumeros(scanner, numeros, cantidad);
mostrarResultados(numeros);

scanner.close();
}

// Solicita al usuario la cantidad de números a ingresar
private static int pedirCantidad(Scanner scanner) {
    System.out.print("Cuántos numeros ingresara? ");
    return scanner.nextInt();
}

// Lee los números ingresados por el usuario
private static void leerNumeros(Scanner scanner, BadNames numeros, int cantidad) {
    for (int i = 0; i < cantidad; i++) {
        System.out.print("Numero: ");
        int numero = scanner.nextInt();
        numeros.agregar(numero);
    }
}

// Muestra la suma y el promedio de los números
private static void mostrarResultados(BadNames numeros) {
    System.out.println("Suma = " + numeros.suma());
    System.out.println("Promedio = " + numeros.promedio());
}
}
```

CONCLUSIONES

1. La elección de nombres significativos facilita la comprensión y reduce la ambigüedad del código

Una de las enseñanzas más relevantes del capítulo 2 es que los nombres deben reflejar con precisión la intención del programador. En el ejemplo analizado, el método denominado `c()` realiza el cálculo del promedio, pero su nombre no proporciona ninguna pista sobre dicha funcionalidad. En cambio, un nombre como `calcularPromedio()` comunica de manera clara su propósito, lo que facilita la comprensión del código por parte de otros desarrolladores y contribuye a un software más mantenible y colaborativo.

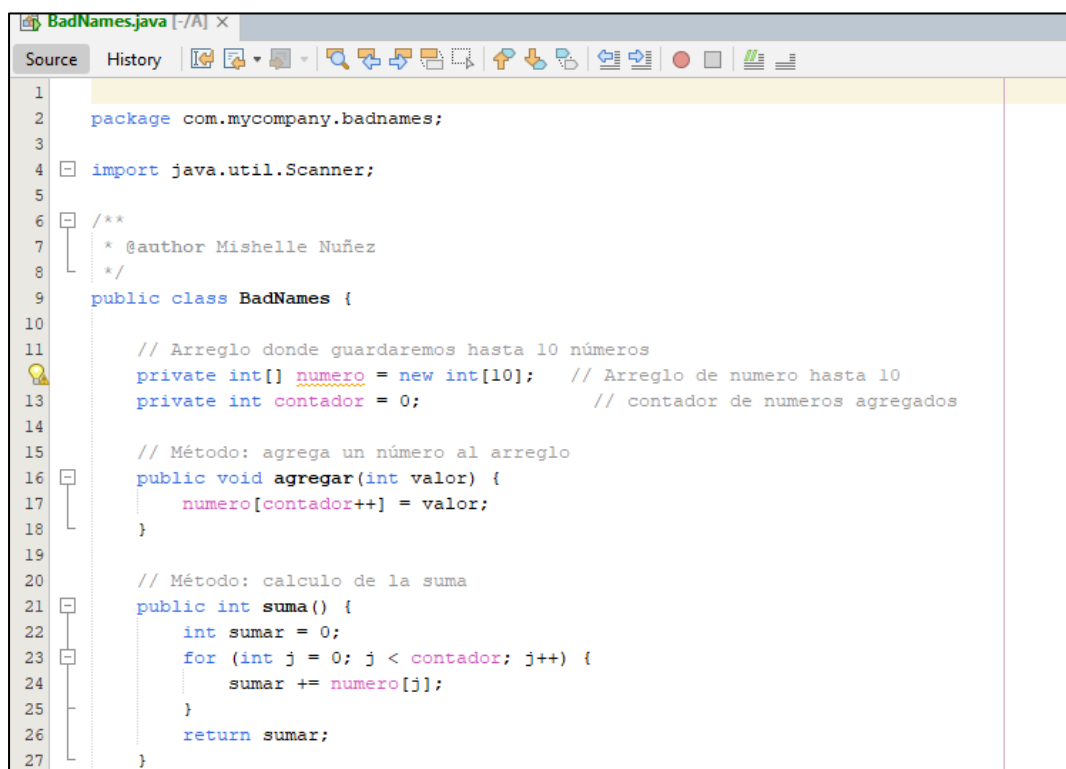
2. Las funciones bien diseñadas deben ser breves, específicas y con un único propósito claro

El capítulo 3 enfatiza la importancia de que una función realice una única tarea y que su nombre lo exprese de forma inequívoca. En el caso del método `c()`, no solo su nombre es confuso, sino que también depende de otro método `b()` (también mal nombrado), lo que complica su interpretación. Al refactorizarlo en funciones como `calcularSuma()` y `calcularPromedio()`, se logra una separación clara de responsabilidades, se mejora la legibilidad y se promueve la reutilización del código.

3. Los nombres crípticos y genéricos aumentan el riesgo de errores y dificultan el mantenimiento del sistema

El uso de identificadores como `a()`, `b()`, `c()`, `i` o `v`, como se observa en el código original, constituye una práctica que va en contra de los principios del *código limpio*. Estos nombres carecen de contexto y obligan al lector a examinar detalladamente la implementación para inferir su propósito. Esta falta de claridad puede provocar errores de uso, reducir la eficiencia del equipo de desarrollo y entorpecer la evolución del software. Por ello, es fundamental adoptar una convención de nombres explícita, descriptiva y coherente.

ANEXOS



```
1 package com.mycompany.badnames;
2
3
4 import java.util.Scanner;
5
6 /**
7  * @author Mishelle Nuñez
8  */
9 public class BadNames {
10
11     // Arreglo donde guardaremos hasta 10 números
12     private int[] numero = new int[10]; // Arreglo de numero hasta 10
13     private int contador = 0;           // contador de numeros agregados
14
15     // Método: agrega un número al arreglo
16     public void agregar(int valor) {
17         numero[contador++] = valor;
18     }
19
20     // Método: calculo de la suma
21     public int suma() {
22         int sumar = 0;
23         for (int j = 0; j < contador; j++) {
24             sumar += numero[j];
25         }
26         return sumar;
27     }
28 }
```

Figura 01: Código corregido en base al capítulo dos.


```

BadNames.java [-/A] x
Source History
29 // Método: promedio de los números guardados
30 public double promedio() {
31     return contador == 0 ? 0 : (double) suma() / contador;
32 }
33
34 // Pequeña interfaz de consola para ejecutar y probar
35 public static void main(String[] args) {
36     BadNames numeros = new BadNames();
37     Scanner scanner = new Scanner(System.in);
38
39     System.out.print("Cuántos numeros ingresara? ");
40     int total = scanner.nextInt();
41
42     for (int k = 0; k < total; k++) { // "k" -> contador genérico
43         System.out.print("Numero: ");
44         numeros.agregar(scanner.nextInt());
45     }
46
47     System.out.println("Suma = " + numeros.suma());
48     System.out.println("Promedio = " + numeros.promedio());
49     scanner.close();
50 }
51 }

```

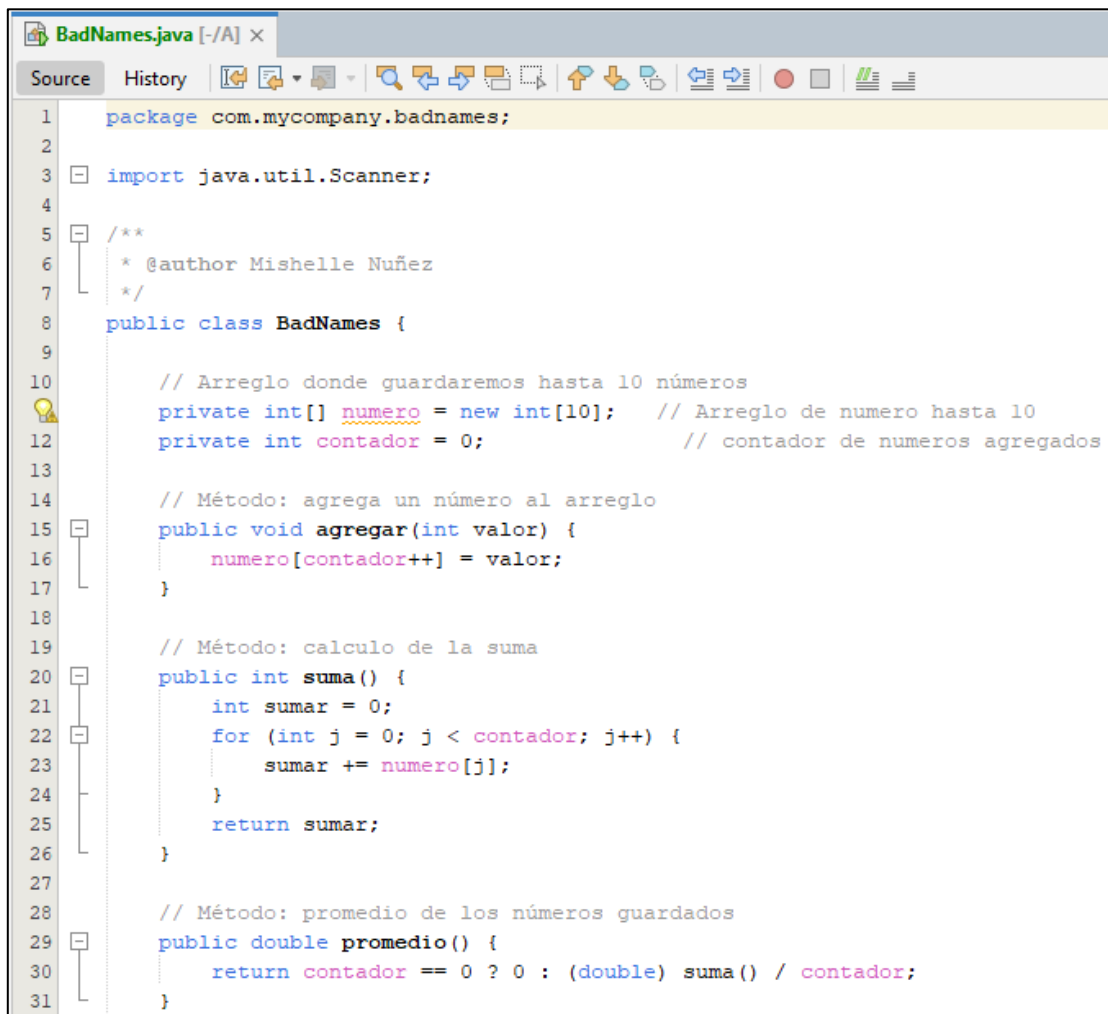
Figura 02: Código corregido en base al capítulo dos.

```

--- exec:3.1.0:exec (default-cli) @ BadNames ---
Cuántos numeros ingresara? 3
Numero: 5
Numero: 7
Numero: 9
Suma = 21
Promedio = 7.0

```

Figura 03: compilación del código corregido en base al capítulo dos.



```
1 package com.mycompany.badnames;
2
3 import java.util.Scanner;
4
5 /**
6  * @author Mishelle Nuñez
7  */
8 public class BadNames {
9
10     // Arreglo donde guardaremos hasta 10 números
11     private int[] numero = new int[10]; // Arreglo de numero hasta 10
12     private int contador = 0;           // contador de numeros agregados
13
14     // Método: agrega un número al arreglo
15     public void agregar(int valor) {
16         numero[contador++] = valor;
17     }
18
19     // Método: calculo de la suma
20     public int suma() {
21         int sumar = 0;
22         for (int j = 0; j < contador; j++) {
23             sumar += numero[j];
24         }
25         return sumar;
26     }
27
28     // Método: promedio de los números guardados
29     public double promedio() {
30         return contador == 0 ? 0 : (double) suma() / contador;
31     }
32 }
```

Figura 01: Código corregido en base al capítulo tres.

```

BadNames.java [-/A] x
Source History
33 // Pequeña interfaz de consola para ejecutar y probar
34 public static void main(String[] args) {
35     BadNames numeros = new BadNames();
36     Scanner scanner = new Scanner(System.in);
37
38     int cantidad = pedirCantidad(scanner);
39     leerNumeros(scanner, numeros, cantidad);
40     mostrarResultados(numeros);
41
42     scanner.close();
43 }
44
45 // Solicita al usuario la cantidad de números a ingresar
46 private static int pedirCantidad(Scanner scanner) {
47     System.out.print("Cuántos numeros ingresara? ");
48     return scanner.nextInt();
49 }

```

Figura 02: Código corregido en base al capítulo tres.

```

51 // Lee los números ingresados por el usuario
52 private static void leerNumeros(Scanner scanner, BadNames numeros, int cantidad) {
53     for (int i = 0; i < cantidad; i++) {
54         System.out.print("Numero: ");
55         int numero = scanner.nextInt();
56         numeros.agregar(numero);
57     }
58 }
59
60 // Muestra la suma y el promedio de los números
61 private static void mostrarResultados(BadNames numeros) {
62     System.out.println("Suma = " + numeros.suma());
63     System.out.println("Promedio = " + numeros.promedio());
64 }
65 }

```

Figura 03: Código corregido en base al capítulo tres.

```

--- exec:3.1.0:exec (default-cli) @ BadNames ---
Cuántos numeros ingresara? 4
Numero: 2
Numero: 3
Numero: 1
Numero: 4
Suma = 10
Promedio = 2.5

```

Figura 04: compilación del código corregido en base al capítulo tres.

BIBLIOGRAFÍA

Martin, R. C. (2009). *Código limpio: Manual de estilo para el desarrollo ágil de software* (pp. 41–86). [PDF]. Pearson Educación.